

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление 02.03.01 Математика и компьютерные науки

Lessons and Exams Timetable Bot

Требования

Гладков И.А., Саржан А.А., Смирнов С.Р., Фролов К.А.

Санкт-Петербург, 2026

Содержание

1	Введение	5
1.1	Основные функции	5
2	Общие требования	7
2.1	Управление версиями и совместная работа	7
2.2	Целевые артефакты	7
3	Функциональные требования	8
3.1	Аутентификация пользователей	8
3.2	Роли пользователей	8
3.3	Авторизация администратора	8
3.4	Служебные команды и эндпоинты	9
3.4.1	Healthcheck	9
3.4.2	Получение списка пользователей	9
4	Подробное описание функций	10
4.1	Запуск бота	10
4.2	Меню	10
5	Расписание занятий и экзаменов	12
5.1	Импорт расписания	12
5.1.1	Поддерживаемые источники	12
5.1.2	Формат CSV	12
5.1.3	Корректный ввод	13
5.1.4	Некорректный ввод	13
5.1.5	Реакция системы	13
5.2	Просмотр расписания на сегодня	14
5.3	Просмотр расписания на завтра	14
5.4	Просмотр расписания на неделю	14
5.5	Добавление занятия	15
5.6	Добавление экзамена	16
5.7	Редактирование события	16
5.8	Удаление события	17
6	Управление учебными задачами	18
6.1	Меню задач	18
6.2	Добавление задачи	18
6.3	Просмотр задач на сегодня	19
6.4	Просмотр задач на завтра	20

6.5	Просмотр задач на неделю	20
6.6	Просмотр задач по предмету	20
6.7	Отметка задачи как выполненной	20
7	Напоминания	21
7.1	Общие требования к напоминаниям	21
7.2	Просмотр настроек напоминаний	21
7.3	Изменение напоминаний	21
7.4	Напоминание о занятии	22
7.5	Напоминание об экзамене	22
7.6	Напоминание о дедлайне	22
8	LLM-советник	24
8.1	Общие требования	24
8.2	Меню LLM-советника	24
8.3	Рекомендация по учебным приоритетам	24
8.4	Генерация плана подготовки к экзамену	25
8.5	Свободный вопрос LLM-советнику	26
9	Дополнительные функции	27
9.1	Интеграция с календарями	27
9.2	Учет учебного времени	27
9.3	LLM-анализ учебных привычек	28
10	Ограничения и лимиты	29
11	Нефункциональные требования	30
11.1	Производительность	30
11.2	Надежность	30
11.3	Валидация ввода	30
11.4	Безопасность	31
11.5	Логирование	31
11.6	Обработка ошибок	32
12	Требования к технологическому стеку	33
13	Требования к документации	33
14	Руководство по сдаче проекта	35
14.1	Требования	35
14.2	Архитектура	35

14.3	Полный проект	35
15	Сценарии использования	36
15.1	Импорт расписания из файла	36
15.2	Просмотр расписания на сегодня	36
15.3	Добавление задачи	36
15.4	Получение рекомендации по приоритетам	37
15.5	Получение напоминания о дедлайне	37
16	Глоссарий	38

1 Введение

Целью проекта является разработка Telegram-бота для управления учебным расписанием, экзаменами, учебными задачами и напоминаниями. Бот должен позволять пользователю импортировать расписание из файла или внешнего источника, просматривать занятия и экзамены на сегодня, завтра и текущую неделю, вручную добавлять и изменять события, вести список учебных задач и получать рекомендации от LLM-советника по учебным приоритетам.

Система предназначена для студентов, которым необходимо централизованно хранить расписание занятий, экзамены, дедлайны и получать напоминания о важных учебных событиях.

1.1 Основные функции

1. Управление расписанием.

- импорт расписания из файла CSV, iCal или внешнего API;
- просмотр расписания на сегодня, завтра и текущую неделю;
- ручное добавление занятий и экзаменов;
- редактирование занятий и экзаменов;
- удаление занятий и экзаменов.

2. Управление учебными задачами.

- добавление задач: упражнения, лабораторные, курсовые, дедлайны;
- привязка задач к учебным предметам;
- просмотр задач по предмету;
- просмотр задач по дедлайну: сегодня, завтра, неделя;
- отметка задачи как выполненной.

3. Напоминания.

- напоминания о предстоящих занятиях;
- напоминания о предстоящих экзаменах;
- напоминания о приближающихся дедлайнах задач.

4. LLM-советник.

- рекомендации по учебным приоритетам;
- генерация плана подготовки к экзамену;
- ответы на свободные вопросы пользователя по учебному планированию.

5. Дополнительные функции.

- интеграция с Google Calendar или Yandex Calendar;
- синхронизация расписания и задач с внешним календарем;
- учет времени обучения по предметам;
- LLM-анализ учебных привычек и рекомендации по улучшению.

2 Общие требования

2.1 Управление версиями и совместная работа

1. Весь код проекта должен быть размещен в репозитории GitHub.
2. Репозиторий должен содержать файл README.md с инструкцией по:
 - настройке проекта локально;
 - сборке и запуску приложения;
 - запуску приложения с использованием Docker;
 - настройке переменных окружения;
 - подключению Telegram-бота;
 - использованию основных команд бота;
 - запуску тестов.
3. История коммитов должна отражать вклад всех участников команды.
4. В репозитории должны отсутствовать секретные данные: токены Telegram, ключи LLM API, ключи календарных API, логины, пароли и строки подключения к базе данных.

2.2 Целевые артефакты

1. Приложение должно собираться в исполняемый **fat JAR**.
2. Приложение должно быть развертываемо в виде Docker-контейнера.
3. Для локального запуска должен быть предоставлен файл docker-compose.yml.
4. Docker Compose должен запускать:
 - приложение Telegram-бота;
 - базу данных;
 - при необходимости - дополнительные сервисы, например брокер сообщений или планировщик задач.
5. Docker-образ приложения должен быть опубликован в Docker Hub или другом согласованном контейнерном registry.

3 Функциональные требования

3.1 Аутентификация пользователей

1. Пользователь идентифицируется автоматически по Telegram chat_id или telegram_id.
2. Дополнительная регистрация пользователя не требуется.
3. При первом запуске бота создается запись пользователя в базе данных.
4. Для каждого пользователя хранятся:
 - Telegram ID;
 - username, если он доступен;
 - настройки часового пояса;
 - настройки напоминаний;
 - расписание;
 - задачи;
 - история учебных сессий, если реализована бонусная функция.

3.2 Роли пользователей

Система должна поддерживать две роли:

1. USER - обычный пользователь Telegram-бота.
2. ADMIN - администратор, имеющий доступ к служебным HTTP-эндпоинтам.

3.3 Авторизация администратора

1. Пользовательская часть работает через Telegram и не требует ручного ввода логина и пароля.
2. Административная часть должна быть доступна через HTTP API.
3. Для доступа к административным эндпоинтам используется API-ключ или Beamer-токен.
4. API-ключ администратора должен выдаваться через эндпоинт /auth при передаче корректных данных администратора.
5. Секретные данные администратора должны храниться в защищенном виде.

3.4 Служебные команды и эндпоинты

3.4.1 Healthcheck

Команда или эндпоинт:

/healthcheck GET /healthcheck

Авторизация не требуется. Система должна возвращать:

- состояние сервера;
- состояние подключения к базе данных;
- версию приложения;
- список авторов проекта.

Пример ответа:

Status: OK Database: OK Version: 1.0.0 Authors: Гладков И.А., Саржан А.А., Смирнов С.Р.,
Фролов К.А.

3.4.2 Получение списка пользователей

HTTP-эндпоинт:

GET /users

Доступен только пользователю с ролью ADMIN. Система должна возвращать список пользователей с их Telegram ID и базовой служебной информацией.

4 Подробное описание функций

4.1 Запуск бота

Команда:

`/start`

Пользователь запускает Telegram-бота командой `/start`.

Реакция системы:

1. Бот определяет пользователя по Telegram ID.
2. Если пользователь новый, создается запись в базе данных.
3. Бот отправляет приветственное сообщение и список доступных команд.

Пример сообщения:

Привет, {username}! Я помогу тебе управлять расписанием занятий, экзаменами, учебными задачами и дедлайнами.

Доступные команды:

`/menu` - открыть меню

`/today` - расписание на сегодня

`/tomorrow` - расписание на завтра

`/week` - расписание на неделю

`/import_timetable` - импорт расписания

`/add_lesson` - добавить занятие

`/add_exam` - добавить экзамен

`/tasks` - учебные задачи

`/reminders` - напоминания

`/advisor` - LLM-советник

`/help` - справка

4.2 Меню

Команда:

`/menu`

Бот выводит основное меню с кнопками:

1. Расписание.
2. Задачи.
3. Напоминания.
4. LLM-советник.

5. Импорт расписания.

6. Настройки.

7. Справка.

Команды меню могут быть реализованы как Telegram-команды или inline-кнопки.

5 Расписание занятий и экзаменов

5.1 Импорт расписания

Команда:

```
/import_timetable
```

Пользователь отправляет команду `/import_timetable`, после чего загружает файл с расписанием или указывает ссылку на внешний источник.

5.1.1 Поддерживаемые источники

Система должна поддерживать хотя бы один из следующих способов импорта:

1. CSV-файл.
2. iCal-файл.
3. Внешний API учебного заведения.

Желательно поддержать CSV и iCal как базовые форматы.

5.1.2 Формат CSV

CSV-файл должен содержать следующие поля:

type, subject, title, start_datetime, end_datetime, location, teacher, comment

Описание полей:

- type - тип события: lesson или exam;
- subject - название предмета;
- title - название занятия или экзамена;
- start_datetime - дата и время начала;
- end_datetime - дата и время окончания;
- location - аудитория или ссылка на онлайн-занятие;
- teacher - преподаватель;
- comment - дополнительная информация.

Пример:

lesson, Математический анализ, Лекция, 12.05.2026 10:00, 12.05.2026 11:30, ауд. 305, Иванов И.И.

Пример:

exam, Алгоритмы и структуры данных, Экзамен, 20.06.2026 09:00, 20.06.2026 12:00, ауд. 101, Петров П.П., Взять зачётку

5.1.3 Корректный ввод

Файл считается корректным, если:

- имеет поддерживаемый формат;
- содержит обязательные поля;
- даты и время указаны в поддерживаемом формате;
- время окончания события позже времени начала;
- тип события равен lesson или exam.

5.1.4 Некорректный ввод

Файл считается некорректным, если:

- формат файла не поддерживается;
- отсутствуют обязательные поля;
- дата или время не распознаются;
- событие содержит некорректный тип;
- время окончания раньше времени начала;
- файл пустой.

5.1.5 Реакция системы

Если импорт успешен:

Расписание успешно импортировано.

Добавлено событий: {count}.

Занятий: {lessons_count}.

Экзаменов: {exams_count}.

Если часть записей некорректна:

Импорт выполнен частично.

Добавлено событий: {count}.

Пропущено строк: {skipped_count}.

Причины ошибок доступны в отчёте.

Если импорт невозможен:

Не удалось импортировать расписание.

Проверьте формат файла и повторите попытку.

5.2 Просмотр расписания на сегодня

Команда:

/today

Бот выводит список занятий и экзаменов пользователя на текущую дату. События должны быть отсортированы по времени начала.

Формат ответа:

Расписание на сегодня, {date}:

1. 10:00-11:30

Математический анализ

Тип: Лекция

Место: ауд. 305

Преподаватель: Иванов И.И.

2. 13:00-14:30

Алгоритмы и структуры данных

Тип: Практика

Место: ауд. 204

Преподаватель: Петров П.П.

Если событий нет:

На сегодня занятий и экзаменов нет.

5.3 Просмотр расписания на завтра

Команда:

/tomorrow

Бот выводит список занятий и экзаменов на следующий календарный день. Если событий нет, бот отправляет сообщение:

На завтра занятий и экзаменов нет.

5.4 Просмотр расписания на неделю

Команда:

/week

Бот выводит расписание на текущую неделю. Текущая неделя считается от понедельника до воскресенья, если иное не задано в настройках пользователя. События группируются по датам.

Пример ответа:

Расписание на неделю:

Понедельник, 11.05.2026 10:00-11:30 - Математический анализ, лекция, ауд. 305

Среда, 13.05.2026 13:00-14:30 - Базы данных, практика, ауд. 212

Пятница, 15.05.2026 09:00-10:30 - Английский язык, семинар, ауд. 110

Если событий нет:

На этой неделе занятий и экзаменов нет.

5.5 Добавление занятия

Команда:

/add_lesson

Пользователь вводит команду /add_lesson и по шагам заполняет данные занятия.

Обязательные поля:

- предмет;
- тип занятия;
- дата;

Необязательные поля:

- время начала;
- время окончания.
- аудитория или ссылка;
- преподаватель;
- комментарий.

Пример корректного ввода:

Предмет: Математический анализ

Тип: Лекция

Дата: 12.05.2026

Время начала: 10:00

Время окончания: 11:30

Место: ауд. 305

Преподаватель: Иванов И.И.

При успешном добавлении:

Занятие успешно добавлено в расписание.

Математический анализ

Лекция

12.05.2026, 10:00-11:30

Место: ауд. 305

При некорректном вводе:

Не удалось добавить занятие. Проверьте дату, время и обязательные поля.

5.6 Добавление экзамена

Команда:

/add_exam

Обязательные поля:

- предмет;
- дата;

Необязательные поля:

- время начала;
- место проведения.
- время окончания;
- преподаватель или экзаменатор;
- комментарий;
- список тем;
- ссылка на материалы для подготовки.

При успешном добавлении:

Экзамен успешно добавлен.

Предмет: Алгоритмы и структуры данных

Дата: 20.06.2026

Время: 09:00

Место: ауд. 101

5.7 Редактирование события

Команда:

/edit_event

или отдельные команды:

/edit_lesson /edit_exam

Пользователь выбирает событие из списка или вводит его идентификатор.

Реакция системы:

1. Бот показывает текущие данные события.
2. Бот предлагает выбрать поле для изменения:

- предмет;
- дата;
- время;
- место;
- преподаватель;
- комментарий.

3. Пользователь вводит новое значение.
4. Система валидирует данные.
5. Событие обновляется.

При успехе:

Событие успешно обновлено.

Если событие не найдено:

Событие не найдено. Проверьте выбранный идентификатор.

5.8 Удаление события

Команда:

`/delete_event`

или отдельные команды:

`/delete_lesson /delete_exam`

Реакция системы:

1. Бот показывает ближайшие события пользователя.
2. Пользователь выбирает событие.
3. Бот запрашивает подтверждение удаления.
4. После подтверждения событие удаляется.

При успехе:

Событие удалено из расписания.

При отмене:

Удаление отменено.

6 Управление учебными задачами

6.1 Меню задач

Команда:

/tasks

Бот показывает меню задач:

Управление задачами:

/add_task - добавить задачу

/tasks_today - задачи на сегодня

/tasks_tomorrow - задачи на завтра

/tasks_week - задачи на неделю

/tasks_by_subject - задачи по предмету

/done_task - отметить задачу выполненной

6.2 Добавление задачи

Команда:

/add_task

Обязательные поля:

- название задачи;
- предмет;

Необязательные поля:

- дедлайн.
- описание;
- тип задачи;
- приоритет;
- примерное время выполнения.

Типы задач:

- упражнение;
- лабораторная работа;
- домашнее задание;
- курсовая работа;

- подготовка к экзамену;
- чтение материалов;
- другое.

Пример корректного ввода:

Название: Решить задачи 1-10

Предмет: Математический анализ

Тип: Домашнее задание

Дедлайн: 15.05.2026 23:59

Приоритет: высокий

Описание: задачи из методички, страница 42

При успехе:

Задача успешно добавлена.

Решить задачи 1-10

Предмет: Математический анализ

Дедлайн: 15.05.2026 23:59

Приоритет: высокий

При ошибке:

Не удалось добавить задачу. Проверьте название, предмет и дедлайн.

6.3 Просмотр задач на сегодня

Команда:

/tasks_today

Бот выводит все невыполненные задачи, дедлайн которых наступает сегодня.

Формат ответа:

Задачи на сегодня:

1. Математический анализ

Решить задачи 1-10

Дедлайн: 23:59

Приоритет: высокий

2. Базы данных

Подготовить ER-диаграмму

Дедлайн: 18:00

Приоритет: средний

Если задач нет:

На сегодня задач нет.

6.4 Просмотр задач на завтра

Команда:

`/tasks_tomorrow`

Бот выводит все невыполненные задачи с дедлайном на завтра. Если задач нет:

На завтра задач нет.

6.5 Просмотр задач на неделю

Команда:

`/tasks_week`

Бот выводит задачи, дедлайн которых находится в пределах текущей недели. Задачи должны быть отсортированы по дедлайну и приоритету. Если задач нет:

На этой неделе задач нет.

6.6 Просмотр задач по предмету

Команда:

`/tasks_by_subject`

Пользователь вводит название предмета или выбирает его из списка. Бот выводит невыполненные задачи по выбранному предмету.

Если предмет не найден:

Предмет не найден. Проверьте название или добавьте задачу/занятие по этому предмету.

Если задач по предмету нет:

По предмету `subject` активных задач нет.

6.7 Отметка задачи как выполненной

Команда:

`/done_task`

Реакция системы:

1. Бот показывает список активных задач.
2. Пользователь выбирает задачу.
3. Система меняет статус задачи на DONE.

При успехе

Задача отмечена как выполненная.

Если задача не найдена:

Задача не найдена или уже выполнена.

7 Напоминания

7.1 Общие требования к напоминаниям

1. Система должна отправлять пользователю Telegram-уведомления о предстоящих занятиях, экзаменах и дедлайнах.
2. Пользователь должен иметь возможность включать и отключать напоминания.
3. Напоминания должны учитывать часовой пояс пользователя.
4. По умолчанию должны быть включены следующие напоминания:
 - за 30 минут до занятия;
 - за 1 день до экзамена;
 - за 2 часа до дедлайна задачи.

Значения по умолчанию могут быть изменены в настройках.

7.2 Просмотр настроек напоминаний

Команда:

`/reminders`

Бот выводит текущие настройки:

Настройки напоминаний:

Занятия: за 30 минут

Экзамены: за 1 день

Задачи: за 2 часа

Статус: включены

Команды:

`/set_reminder` - изменить настройки

`/disable_reminders` - отключить напоминания

`/enable_reminders` - включить напоминания

7.3 Изменение напоминаний

Команда:

`/set_reminder`

Бот предлагает выбрать тип напоминания:

1. Занятие.
2. Экзамен.
3. Задача.

После выбора пользователь вводит время до события.

Пример корректного ввода:

30m

2h

1d

Где:

- m - минуты;
- h - часы;
- d - дни.

При успехе:

Настройки напоминания обновлены.

При ошибке:

Некорректный формат времени. Используйте формат 30m, 2h или 1d.

7.4 Напоминание о занятии

Система должна отправлять сообщение:

Напоминание о занятии:

Через 30 минут начнётся занятие:

Математический анализ

Тип: Лекция

Время: 10:00-11:30

Место: ауд. 305

7.5 Напоминание об экзамене

Система должна отправлять сообщение:

Напоминание об экзамене:

Завтра экзамен:

Алгоритмы и структуры данных

Дата: 20.06.2026

Время: 09:00

Место: ауд. 101

7.6 Напоминание о дедлайне

Система должна отправлять сообщение:

Напоминание о дедлайне:

До дедлайна осталось 2 часа.

Задача: Подготовить ER-диаграмму

Предмет: Базы данных

Дедлайн: 18:00

8 LLM-советник

8.1 Общие требования

1. Бот должен использовать внешний LLM API для генерации учебных рекомендаций.
2. LLM-советник должен использовать данные пользователя:
 - расписание занятий;
 - даты экзаменов;
 - учебные задачи;
 - дедлайны;
 - приоритеты задач;
 - историю учебных сессий, если реализована бонусная функция.
3. Ответы LLM должны носить рекомендательный характер.
4. Если данных недостаточно, бот должен сообщить пользователю, какие данные нужно добавить.

Пример сообщения при недостатке данных:

Недостаточно данных для рекомендации. Добавьте задачи или экзамены, чтобы я мог оценить приоритеты.

8.2 Меню LLM-советника

Команда:

/advisor

Бот выводит меню:

LLM-советник:

/priority - что учить в первую очередь

/exam_plan - план подготовки к экзамену

/ask_advisor - задать вопрос советнику

8.3 Рекомендация по учебным приоритетам

Команда:

/priority

Система анализирует:

- ближайшие дедлайны;
- даты экзаменов;

- количество невыполненных задач;
- приоритет задач;
- свободные временные окна в расписании.

Пример ответа:

Рекомендация по приоритетам:

1. Базы данных - высокий приоритет

Причина: дедлайн по ER-диаграмме сегодня в 18:00.

2. Алгоритмы и структуры данных - высокий приоритет

Причина: экзамен через 5 дней, есть 3 незавершённые задачи.

3. Математический анализ - средний приоритет

Причина: ближайший дедлайн через 4 дня.

8.4 Генерация плана подготовки к экзамену

Команда:

/exam_plan

Пользователь выбирает экзамен из списка или вводит название предмета. Бот формирует план подготовки на основе:

- даты экзамена;
- оставшегося количества дней;
- списка задач по предмету;
- свободных окон в расписании;
- указанной пользователем сложности предмета, если такая информация есть.

Пример ответа:

План подготовки к экзамену по предмету "Алгоритмы и структуры данных":

День 1: Повторить сортировки и асимптотику.

Время: 1.5 часа.

День 2: Разобрать деревья поиска и хеш-таблицы.

Время: 2 часа.

День 3: Решить тренировочные задачи.

Время: 2 часа.

День 4: Повторить конспекты и слабые темы.

Время: 1 час.

День 5: Лёгкое повторение, без перегрузки.

Время: 1 час.

Если экзамен не найден:

Экзамен по указанному предмету не найден. Добавьте экзамен через /add_exam.

8.5 Свободный вопрос LLM-советнику

Команда:

/ask_advisor

Пользователь задает вопрос в свободной форме.

Примеры:

- Как распределить подготовку к двум экзаменам?
- Что лучше учить сегодня?
- Как не забыть про дедлайны на этой неделе?

Реакция системы:

1. Бот получает вопрос пользователя.
2. Бот добавляет в контекст расписание, задачи и экзамены пользователя.
3. Бот отправляет запрос к LLM.
4. Бот возвращает ответ пользователю.

Если LLM API недоступен:

LLM-советник временно недоступен. Попробуйте повторить запрос позже.

9 Дополнительные функции

9.1 Интеграция с календарями

В качестве бонусной функции система может поддерживать интеграцию с:

- Google Calendar;
- Yandex Calendar;
- другим календарным API, доступным в учебном заведении.

Система должна позволять:

- экспортировать занятия в календарь;
- экспортировать экзамены в календарь;
- экспортировать дедлайны задач в календарь;
- синхронизировать изменения расписания;
- отключать синхронизацию.

Команды:

/calendar_sync
/calendar_export
/calendar_disable

При успешной синхронизации:

Синхронизация с календарём выполнена. Добавлено событий: count.

При ошибке авторизации:

Не удалось подключиться к календарю. Проверьте доступы и повторите попытку.

9.2 Учет учебного времени

Команды:

/study_start
/study_stop
/study_stats

Пользователь может запускать и завершать учебную сессию по конкретному предмету.

Сценарий:

1. Пользователь вводит команду /study_start.
2. Бот запрашивает предмет.
3. Пользователь выбирает предмет.

4. Бот фиксирует начало учебной сессии.

Пример ответа:

Учебная сессия начата.

Предмет: Базы данных

Начало: 16:00

5. Пользователь завершает сессию командой `/study_stop`. Бот сохраняет длительность.

Пример ответа:

Учебная сессия завершена.

Предмет: Базы данных

Длительность: 1 час 20 минут

Команда `/study_stats` выводит:

- общее время обучения за день;
- общее время обучения за неделю;
- распределение времени по предметам.

9.3 LLM-анализ учебных привычек

Команда:

`/habits_analysis`

Если реализован учет учебного времени, бот может анализировать учебные привычки пользователя. Система передает в LLM:

- историю учебных сессий;
- задачи;
- дедлайны;
- расписание;
- экзамены.

Пример ответа:

Анализ учебных привычек:

1. Ты чаще занимаешься вечером, но дедлайны часто закрываются в последний день.
2. Рекомендуется начинать подготовку к крупным задачам минимум за 3 дня.
3. На этой неделе больше всего времени ушло на Базы данных, но экзамен по Алгоритмам ближе.
4. Совет: выдели завтра 1.5 часа на Алгоритмы и 1 час на Математический анализ.

10 Ограничения и лимиты

1. Один пользователь не должен иметь более одного активного процесса импорта расписания одновременно.
2. Максимальный размер импортируемого файла - 5 МБ.
3. Максимальное количество событий расписания на одного пользователя - 5000.
4. Максимальное количество активных задач на одного пользователя - 1000.
5. Максимальная длина названия предмета - 100 символов.
6. Максимальная длина описания задачи - 1000 символов.
7. Максимальная длина вопроса к LLM-советнику - 2000 символов.
8. Бот должен поддерживать не менее 1000 активных пользователей.
9. Расписание на неделю должно выводиться не более чем за 2 секунды без учета задержек внешних API.
10. При недоступности внешнего календарного API система должна сохранять данные локально и сообщать пользователю о проблеме синхронизации.

11 Нефункциональные требования

11.1 Производительность

1. Максимальное время ответа на простые команды, такие как `/menu`, `/today`, `/tasks_today`, не должно превышать 500 мс без учета задержек внешних API.
2. Импорт расписания должен выполняться не дольше 10 секунд для файла до 1000 событий.
3. Запрос к LLM должен иметь таймаут не более 30 секунд.
4. При долгих операциях бот должен сообщать пользователю, что запрос принят в обработку.

11.2 Надежность

1. При ошибках внешнего API система должна выполнять повторные попытки.
2. Количество повторных попыток - не более 3.
3. Интервал между повторными попытками - 1 секунда для обычных API и до 30 секунд для LLM API.
4. Если внешний API недоступен после всех попыток, пользователь должен получить понятное сообщение об ошибке.
5. Все данные пользователя должны сохраняться в базе данных.
6. Система не должна терять уже сохраненное расписание при неудачном импорте нового файла.

11.3 Валидация ввода

Система должна проверять:

- корректность дат;
- корректность времени;
- наличие обязательных полей;
- корректность формата файла;
- корректность типа события;
- длину пользовательского ввода;
- принадлежность задачи существующему или новому предмету;
- корректность команд Telegram.

При некорректном вводе бот должен объяснять ошибку и предлагать повторить ввод.

11.4 Безопасность

1. Токен Telegram-бота должен храниться в переменных окружения.
2. Ключи LLM API должны храниться в переменных окружения.
3. Ключи календарных API должны храниться в переменных окружения или защищенном хранилище.
4. Данные подключения к базе данных не должны храниться в коде.
5. Административные эндпоинты должны быть защищены авторизацией.
6. Пользователь не должен иметь доступ к расписанию, задачам и учебным данным других пользователей.
7. Все входные данные должны проходить валидацию для защиты от инъекций и некорректных запросов.

11.5 Логирование

Система должна логировать:

- запуск приложения;
- ошибки Telegram API;
- ошибки базы данных;
- ошибки внешних API;
- ошибки импорта расписания;
- ошибки LLM-запросов;
- административные запросы.

В логах не должны сохраняться:

- токены;
- пароли;
- API-ключи;
- персональные данные в избыточном объеме.

11.6 Обработка ошибок

Все ошибки должны сопровождаться понятным сообщением для пользователя.

Примеры:

- Файл не удалось обработать. Проверьте формат и повторите попытку.
- Событие не найдено. Возможно, оно было удалено ранее.
- LLM-советник временно недоступен. Попробуйте позже.
- Не удалось подключиться к календарю. Проверьте настройки интеграции.

12 Требования к технологическому стеку

Рекомендуемый стек проекта:

- язык программирования: Java 25;
- фреймворк: Spring WebFlux, Spring Data MongoDB Reactive, Spring REST Docs;
- Telegram API: для реализации Telegram-бота;
- база данных: MongoDB;
- брокер сообщений: RabbitMQ;
- инструмент сборки: Maven или Gradle;
- контейнеризация: Docker;
- локальный запуск: Docker Compose;
- тестирование: JUnit;
- документация API: Spring REST Docs или Swagger/OpenAPI;
- внешний LLM API: любой доступный LLM-провайдер;
- календарные интеграции: Google Calendar API, Yandex Calendar API или API учебного заведения.

13 Требования к документации

Документация проекта должна включать:

1. README.md.

- описание проекта;
- список функций;
- список команд Telegram-бота;
- инструкцию по локальному запуску;
- инструкцию по запуску через Docker;
- описание переменных окружения;
- инструкцию по запуску тестов.

2. Документация API.

- описание эндпоинтов /healthcheck, /auth, /users;

- описание формата запросов и ответов;
- описание ошибок.

3. Документация по формату импорта.

- пример CSV-файла;
- описание обязательных полей;
- описание поддерживаемых форматов даты и времени;
- пример iCal-файла или ссылка на поддерживаемый формат.

4. Документация по архитектуре.

- схема компонентов;
- описание модулей системы;
- схема взаимодействия с Telegram API;
- схема взаимодействия с LLM API;
- схема взаимодействия с календарным API, если реализовано.

14 Руководство по сдаче проекта

14.1 Требования

Необходимо предоставить документ с требованиями, включающий:

- цель проекта;
- описание основной функциональности;
- функциональные требования;
- нефункциональные требования;
- ограничения;
- используемый технологический стек;
- глоссарий.

14.2 Архитектура

Необходимо предоставить документ по архитектуре, включающий:

- высокоуровневое описание системы;
- компонентную диаграмму;
- описание базы данных;
- описание взаимодействия с Telegram API;
- описание взаимодействия с LLM API;
- описание работы напоминаний;
- описание развертывания приложения.

14.3 Полный проект

Необходимо предоставить:

- ссылку на GitHub-репозиторий;
- ссылку на Docker Hub с опубликованным образом;
- имя или ссылку на Telegram-бота;
- инструкцию по тестированию;
- примеры файлов для импорта расписания.

15 Сценарии использования

15.1 Импорт расписания из файла

Действия пользователя:

1. Пользователь вводит команду `/import_timetable`.
2. Пользователь отправляет CSV или iCal-файл.

Реакция системы:

1. Бот принимает файл.
2. Проверяет формат.
3. Валидирует строки расписания.
4. Сохраняет корректные события в базу данных.
5. Возвращает отчет об импорте.

Результат: пользователь получает сообщение об успешном или частичном импорте расписания.

15.2 Просмотр расписания на сегодня

Действия пользователя:

1. Пользователь вводит команду `/today`.

Реакция системы:

1. Бот определяет текущую дату с учетом часового пояса пользователя.
2. Загружает занятия и экзамены на текущую дату.
3. Сортирует события по времени.
4. Отправляет пользователю список событий.

Результат: пользователь видит расписание на сегодня.

15.3 Добавление задачи

Действия пользователя:

1. Пользователь вводит команду `/add_task`.
2. Пользователь вводит название задачи.

3. Пользователь выбирает или вводит предмет.
4. Пользователь указывает дедлайн.
5. Пользователь при необходимости указывает приоритет и описание.

Реакция системы:

1. Бот проверяет корректность данных.
2. Сохраняет задачу в базе данных.
3. Подтверждает добавление задачи.

Результат: задача появляется в списке задач пользователя.

15.4 Получение рекомендации по приоритетам

Действия пользователя:

1. Пользователь вводит команду /priority.

Реакция системы:

1. Бот собирает данные о задачах, дедлайнах, экзаменах и расписании.
2. Передает структурированный контекст в LLM.
3. Получает рекомендацию.
4. Отправляет пользователю список приоритетов.

Результат: пользователь получает рекомендацию, какие предметы и задачи нужно выполнить в первую очередь.

15.5 Получение напоминания о дедлайне

Условие: у пользователя есть активная задача с дедлайном.

Реакция системы:

1. Планировщик проверяет ближайшие дедлайны.
2. Если дедлайн приближается, система отправляет сообщение в Telegram.
3. Пользователь получает напоминание.

Результат: пользователь заранее узнает о приближающемся дедлайне.

16 Глоссарий

Термин	Определение
Telegram-бот	Автоматизированная программа, работающая внутри Telegram и обрабатывающая команды пользователя.
Telegram ID	Уникальный идентификатор пользователя в Telegram.
Расписание	Набор занятий и экзаменов пользователя с датой, временем и дополнительными параметрами.
Занятие	Учебное событие: лекция, практика, лабораторная, семинар или другое занятие.
Экзамен	Контрольное учебное событие по предмету, имеющее дату, время и место проведения.
Предмет	Учебная дисциплина, к которой относятся занятия, экзамены и задачи.
Задача	Учебное дело пользователя: домашняя работа, упражнение, курсовая, лабораторная или другой дедлайн.
Дедлайн	Дата и время, к которым задача должна быть выполнена.
Напоминание	Сообщение от бота, отправляемое до наступления занятия, экзамена или дедлайна.
LLM	Большая языковая модель, используемая для генерации учебных рекомендаций.
LLM-советник	Функция бота, которая помогает пользователю определить приоритеты и составить план подготовки.
CSV	Текстовый формат хранения табличных данных, используемый для импорта расписания.
iCal	Формат календарных событий, используемый для обмена расписанием между календарными приложениями.
Google Calendar API	API для интеграции с календарем Google.
Yandex Calendar API	API для интеграции с календарем Яндекса.
Docker	Платформа для контейнеризации приложения.
Docker Compose	Инструмент для запуска нескольких связанных Docker-контейнеров.
Fat JAR	Исполняемый JAR-файл, содержащий приложение и необходимые зависимости.
База данных	Система хранения данных о пользователях, расписании, задачах и напоминаниях.
Healthcheck	Проверка работоспособности приложения и его зависимостей.

Термин	Определение
API-ключ	Секретная строка, используемая для доступа к административным эндпоинтам.
ADMIN	Роль администратора системы.
USER	Стандартная роль пользователя Telegram-бота.
Учебная сессия	Период времени, в течение которого пользователь занимался конкретным предметом.
Синхронизация	Обмен событиями между ботом и внешним календарем.